# PYTHON PROJECT REPORT

# TO-DO LIST AND TASK MANAGER

Programming Language: Python 3
Project Type: Menu-driven console application
Project Level: Beginner

Submitted by: KANIGICHARLA LAKSHMI SUVARNA
Roll Number: 238A1A0518
Department / College: RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS
Academic Year: 2026-27

## 1. Abstract

The To-Do List and Task Manager is a beginner-level Python application that helps users organize and track daily activities. The program provides a menu through which users can add tasks, view all tasks, mark tasks as completed, edit task descriptions, delete tasks, search by task ID or keyword, and view a completion summary. Each task is stored as a Python dictionary, and all task dictionaries are maintained in a list. The project demonstrates fundamental Python concepts, including variables, lists, dictionaries, functions, loops, conditional statements, string methods, and exception handling.

## 2. Problem Statement

Managing daily activities manually can result in forgotten tasks, missed deadlines, and difficulty tracking progress. The project addresses this problem by providing a simple computerized task manager in which users can record tasks, update their status, modify or remove tasks, search for specific items, and review overall completion progress from one menu.

## 3. Objectives

- Add new tasks with unique task IDs.
- Display all tasks and their completion status.
- Mark pending tasks as completed.
- Edit task descriptions and delete unwanted tasks.
- Search tasks using a task ID or keyword.
- Display the total number of tasks, completed tasks, pending tasks, and completion percentage.
- Practice basic Python programming concepts through a practical application.

## 4. Project Scope

The application is intended for personal task management through a console-based interface. It supports adding, viewing, completing, editing, deleting, and searching tasks, along with a completion summary. In the reference implementation, task data is stored temporarily in memory while the program is running. Tasks are not saved permanently after the program exits. File handling or database storage can be added as a future enhancement.

## 5. Technologies Used

| Component | Technology / Description |
|---|---|
| Programming language | Python 3 |
| Development environment | IDLE, Visual Studio Code, or PyCharm |
| Data storage | Python list and dictionaries |
| User interface | Menu-driven console |
| External libraries | Not required |

## 6. Modules and Responsibilities

- **Module 1: Add Tasks:** Accepts a task description, validates that it is not empty, assigns a unique task ID, and stores the new task.
- **Module 2: View All Tasks:** Displays each task's ID, description, and status. If no tasks exist, it displays an appropriate message.

- **Module 3: Mark Tasks as Completed:** Finds a task by ID and changes its status to completed. It handles invalid IDs and tasks that are already completed.
- **Module 4A: Edit a Task:** Allows the user to change the description of an existing task.
- **Module 4B: Delete a Task:** Removes a selected task from the task list using its ID.
- **Module 5A: Search Tasks:** Searches for tasks by exact ID or by a keyword in the task description.
- **Module 5B: Completion Summary:** Calculates total, completed, and pending tasks and the completion percentage.
- **Main Program:** Displays the menu, accepts the user's choice, calls the appropriate function, and repeats until the user exits.

## 7. System Design

### 7.1 Task Data Structure

Each task is represented by a Python dictionary with the following fields:

| Field | Description | Example |
|---|---|---|
| id | Unique task number | 1 |
| description | Text describing the task | Complete Python project |
| completed | Boolean completion status | False |

All task dictionaries are stored in a list named tasks. A separate next_id variable is used to assign task IDs as new tasks are created.

### 7.2 Application Workflow

1. Start the program.
2. Display the main menu.
3. Read the user's menu choice.
4. Perform the selected operation: add, view, complete, edit, delete, search, or summary.
5. Display the result or an error message where necessary.
6. Return to the menu or exit when the user selects 0.

## 8. Complete Python Source Code

Paste the complete source code for the project in this section. The source code is intentionally left for the student to add, as requested.

### 8.1 Source Code

```
# TO-DO LIST AND TASK MANAGER
# Basic Python: Lists, Loops, Conditions and Functions
users = []
tasks = []
sections = ["Personal", "Academic", "Projects"]
next_user_id = 1
next_task_id = 1
# ---------- NAME VALIDATION ----------
def validate_name(name):
    if name == "" or name != name.strip():
        return False
```

```
    if name.startswith(".") or name.endswith("."):
        return False
    if name.count(".") > 1:
        return False
    for ch in name:
        if not (ch.isalpha() or ch == " " or ch == "."):
            return False
    return True


# ---------- PASSWORD VALIDATION ----------

def validate_password(password):
    if len(password) != 10:
        return False
    upper = 0
    lower = 0
    digits = 0
    special = 0
    for ch in password:
        if ch.isupper() and ch.isalpha():
            upper += 1
        elif ch.islower() and ch.isalpha():
            lower += 1
        elif ch.isdigit():
            digits += 1
        else:
            special += 1
    return (
        upper == 1
        and lower == 4
        and digits == 4
        and special == 1
    )


# ---------- REGISTRATION ----------

def register_user():
    global next_user_id
    print("\n========== USER REGISTRATION ==========")
    while True:
        name = input("Enter your name: ").strip()
        if not validate_name(name):
            print("Invalid name!")
            print("Use letters and spaces, with at most one dot.")
            continue
        duplicate = False
        for user in users:
            if user[1].lower() == name.lower():
                duplicate = True
                break
        if duplicate:
            print("This name is already registered. Try another name.")
```

```
        else:
            break
    while True:
        password = input("Create your password: ")
        if validate_password(password):
            break
        print("Invalid password!")
        print("Requirements:")
        print("1 uppercase letter")
        print("4 lowercase letters")
        print("1 special character")
        print("4 digits")
        print("Exactly 10 characters, in any order.")
    user = [next_user_id, name, password]
    users.append(user)
    print("\nRegistration successful!")
    print("Your User ID is:", next_user_id)
    next_user_id += 1


# ---------- LOGIN ----------

def login():
    print("\n========== LOGIN ==========")
    name = input("Enter your name: ").strip()
    password = input("Enter your password: ")
    for user in users:
        if user[1].lower() == name.lower() and user[2] == password:
            print("\nLogin successful!")
            print("Welcome,", user[1])
            return user
    print("Invalid name or password.")
    return None


# ---------- SELECT SECTION ----------

def choose_section():
    print("\nAvailable Sections:")
    for i in range(len(sections)):
        print(i + 1, ".", sections[i])
    while True:
        section = input("Enter section name: ").strip()
        for available in sections:
            if section.lower() == available.lower():
                return available
        print("Invalid section. Please try again.")


# ---------- TASK HELPER FUNCTIONS ----------

def get_user_tasks(user_id):
    result = []
```

```
    for task in tasks:
        if task[1] == user_id:
            result.append(task)
    return result
def find_task(user_id, task_id):
    for task in tasks:
        if task[0] == task_id and task[1] == user_id:
            return task
    return None
def get_task_id():
    while True:
        value = input("Enter task ID: ").strip()
        if value.isdigit():
            return int(value)
        print("Enter a valid numeric ID.")
def generate_task_id():
    global next_task_id
    task_id = next_task_id
    next_task_id += 1
    return task_id


# ---------- ADD TASK ----------

def add_task(user):
    print("\n========== ADD TASK ==========")
    section = choose_section()
    title = input("Enter task title: ").strip()
    if title == "":
        print("Task title cannot be empty.")
        return
    description = input("Enter task description: ").strip()
    task = [
        generate_task_id(),
        user[0],
        section,
        title,
        description,
        "Pending"
    ]
    tasks.append(task)
    print("Task added successfully!")


# ---------- VIEW TASKS ----------

def view_tasks(user):
    print("\n========== YOUR TASKS ==========")
    found = False
    for task in tasks:
        if task[1] == user[0]:
            print("----------------------------------")
            print("Task ID:", task[0])
            print("Section:", task[2])
```

```
            print("Title:", task[3])
            print("Description:", task[4])
            print("Status:", task[5])
            found = True
    if not found:
        print("No tasks found.")


# ---------- VIEW TASKS BY SECTION ----------

def view_section_tasks(user):
    section = choose_section()
    print("\nTasks in", section)
    found = False
    for task in tasks:
        if task[1] == user[0] and task[2] == section:
            print(
                "ID:", task[0],
                "| Title:", task[3],
                "| Status:", task[5]
            )
            found = True
    if not found:
        print("No tasks found in this section.")


# ---------- MARK COMPLETED ----------

def mark_completed(user):
    view_tasks(user)
    task_id = get_task_id()
    task = find_task(user[0], task_id)
    if task is None:
        print("Task not found.")
    elif task[5] == "Completed":
        print("Task is already completed.")
    else:
        task[5] = "Completed"
        print("Task marked as completed!")


# ---------- EDIT TASK ----------

def edit_task(user):
    view_tasks(user)
    task_id = get_task_id()
    task = find_task(user[0], task_id)
    if task is None:
        print("Task not found.")
        return
    title = input(
        "Enter new title (press Enter to keep current): "
    ).strip()
```

```
    if title != "":
        task[3] = title
    description = input(
        "Enter new description (press Enter to keep current): "
    ).strip()
    if description != "":
        task[4] = description
    change = input("Change section? (yes/no): ").strip().lower()
    if change == "yes":
        task[2] = choose_section()
    print("Task updated successfully!")


# ---------- DELETE TASK ----------

def delete_task(user):
    view_tasks(user)
    task_id = get_task_id()
    task = find_task(user[0], task_id)
    if task is None:
        print("Task not found.")
        return
    confirm = input(
        "Are you sure you want to delete this task? (yes/no): "
    ).strip().lower()
    if confirm == "yes":
        tasks.remove(task)
        print("Task deleted successfully!")
    else:
        print("Deletion cancelled.")


# ---------- SEARCH TASKS ----------

def search_tasks(user):
    keyword = input("Enter search keyword: ").strip().lower()
    if keyword == "":
        print("Search keyword cannot be empty.")
        return
    found = False
    for task in tasks:
        if task[1] == user[0]:
            if (keyword in task[3].lower()
                    or keyword in task[4].lower()):
                print(
                    "ID:", task[0],
                    "| Section:", task[2],
                    "| Title:", task[3],
                    "| Status:", task[5]
                )
                found = True
    if not found:
        print("No matching tasks found.")
```

```
# ---------- COMPLETION SUMMARY ----------

def completion_summary(user):
    total = 0
    completed = 0
    pending = 0
    for task in tasks:
        if task[1] == user[0]:
            total += 1
            if task[5] == "Completed":
                completed += 1
            else:
                pending += 1
    percentage = 0
    if total > 0:
        percentage = completed * 100 / total
    print("\n========== COMPLETION SUMMARY ==========")
    print("Total tasks:", total)
    print("Completed tasks:", completed)
    print("Pending tasks:", pending)
    print("Completion rate: {:.2f}%".format(percentage))
    print("\nSection-wise summary:")
    for section in sections:
        section_total = 0
        section_completed = 0
        for task in tasks:
            if task[1] == user[0] and task[2] == section:
                section_total += 1
                if task[5] == "Completed":
                    section_completed += 1
        print(
            section + ":",
            section_completed,
            "completed out of",
            section_total
        )


# ---------- TASK MANAGER MENU ----------

def task_manager(user):
    while True:
        print("\n========== TASK MANAGER ==========")
        print("1. Add task")
        print("2. View all tasks")
        print("3. View tasks by section")
        print("4. Mark task completed")
        print("5. Edit task")
        print("6. Delete task")
        print("7. Search tasks")
        print("8. Completion summary")
        print("9. Logout")
```

```
        choice = input("Enter your choice: ").strip()
        if choice == "1":
            add_task(user)
        elif choice == "2":
            view_tasks(user)
        elif choice == "3":
            view_section_tasks(user)
        elif choice == "4":
            mark_completed(user)
        elif choice == "5":
            edit_task(user)
        elif choice == "6":
            delete_task(user)
        elif choice == "7":
            search_tasks(user)
        elif choice == "8":
            completion_summary(user)
        elif choice == "9":
            print("Logged out successfully.")
            break
        else:
            print("Invalid choice. Enter a number from 1 to 9.")


# ---------- MAIN PROGRAM ----------

def main():
    while True:
        print("\n====================================")
        print("   TO-DO LIST AND TASK MANAGER")
        print("====================================")
        print("1. Register")
        print("2. Login")
        print("3. Exit")
        choice = input("Enter your choice: ").strip()
        if choice == "1":
            register_user()
        elif choice == "2":
            if len(users) == 0:
                print("No registered users. Please register first.")
            else:
                user = login()
                if user is not None:
                    task_manager(user)
        elif choice == "3":
            print("Thank you for using the application!")
            break
        else:
            print("Invalid choice. Enter 1, 2, or 3.")
main()
```

## 9. Sample Output

Example interaction based on the functions described in the reference project:

```
===== TO-DO LIST AND TASK MANAGER =====
1. Add Task
2. View All Tasks
3. Mark Task as Completed
4. Edit Task
5. Delete Task
6. Search Tasks
7. Completion Summary
0. Exit

Enter your choice: 1
Enter task description: Complete Python project
Task added successfully. Task ID: 1

Enter your choice: 3
Enter task ID to complete: 1
Task marked as completed.

--- COMPLETION SUMMARY ---
Total tasks: 1
Completed tasks: 1
Pending tasks: 0
Completion percentage: 100.0 %
```

```
Task ID: 1
Section: Personal
Title: wash cloths
Description: at morning
Status: Pending
-----------------------------------
Task ID: 2
Section: Academic
Title: write assignment
Description: HRPM assignment
Status: Pending
-----------------------------------
Task ID: 3
Section: Projects
Title: code
Description: write code
Status: Pending
Enter task ID: academic
Enter a valid numeric ID.
Enter task ID: 2
Task marked as completed!
```

```
========== TASK MANAGER ==========
1. Add task
2. View all tasks
3. View tasks by section
4. Mark task completed
5. Edit task
6. Delete task
7. Search tasks
8. Completion summary
9. Logout
Enter your choice: 1

========== ADD TASK ==========

Available Sections:
1 . Personal
2 . Academic
3 . Projects
Enter section name: academic
Enter task title: write assignment
Enter task description: HRPM assignment
```

```
========== USER REGISTRATION ==========
Enter your name: name one
Create your password: Nameo@1234

Registration successful!
Your User ID is: 1

====================================
   TO-DO LIST AND TASK MANAGER
====================================
1. Register
2. Login
3. Exit
Enter your choice: 2

========== LOGIN ==========
Enter your name: name one
Enter your password: Nameo@1234

Login successful!
Welcome, name one
```

## 10. How to Run the Project

7. Install Python 3 on the computer.
8. Open IDLE, Visual Studio Code, PyCharm, or another Python editor.
9. Create a Python file named todo_manager.py.
10. Paste the completed project source code into the file.
11. Save the file.
12. Open a terminal in the project folder and run: python todo_manager.py
13. Use the numbered menu options to perform task-management operations.

## 11. Test Cases

| Test | Input / Action | Expected Result |
|---|---|---|
| 1 | Add a valid task | Task is added with a unique ID. |
| 2 | View tasks when the list is empty | Displays 'No tasks available.' |
| 3 | Complete an existing task | Task status changes to Completed. |
| 4 | Complete a task that is already completed | Displays a message that the task is already completed. |
| 5 | Edit an existing task | Task description is updated. |
| 6 | Delete an existing task | Task is removed from the list. |
| 7 | Search using a matching keyword | Matching tasks are displayed. |
| 8 | Search for a task that does not exist | Displays 'No matching tasks found.' |
| 9 | Check summary with two tasks, one completed | Displays 50% completion. |
| 10 | Enter an invalid menu option | Displays 'Invalid choice. Please try again.' |
| 11 | Enter non-numeric task ID input | Displays a message requesting a valid numeric ID. |
| 12 | Enter an empty task description | Rejects the empty description. |

## 12. Python Concepts Learned

- Lists and dictionaries for storing task data.
- Functions and function calls for modular program design.
- while and for loops.
- if, elif, and else conditional statements.
- Input and output using input() and print().
- Searching and updating list elements.
- String methods such as strip() and lower().
- Exception handling using try and except.
- Basic calculations and percentage formulas.

## 13. Project Limitations

- Tasks are stored only in memory.
- All tasks are lost when the program exits.
- The application supports one user at a time.
- The application does not include due dates, reminders, or priority levels.
- The interface is console-based rather than graphical.

## 14. Future Enhancements

14. Use file handling to save tasks permanently.
15. Add due dates and deadline reminders.
16. Add priority levels such as High, Medium, and Low.
17. Organize tasks into categories such as Personal, College, and Work.
18. Create a graphical user interface using Tkinter.
19. Use SQLite or another database for persistent storage.
20. Add user registration and login.

## 15. Conclusion

The To-Do List and Task Manager is a beginner-friendly Python project that demonstrates how core programming concepts can be combined to solve a practical problem. The application lets users maintain a task list, update task status, edit or delete entries, search for tasks, and monitor completion progress. Its modular functions make the program easier to understand, test, and maintain. Future enhancements such as persistent storage, deadlines, priorities, and a graphical interface can make it more useful.